SECURE STEGANOGRAPHY SYSTEM

Group Member

Mariam Mudassar ………….. 70176114
Anum Rafaqat…......………… 70176153
Kainat Bashir ………..……… 70175376
4th Sem-4E

Submitted to : Professor Amjad Raza

Information Security Project Documentation

Table of Contents

Literature Review

Cryptography

Cryptography is the science of protecting information by converting readable data into an unreadable form. It is widely used to secure sensitive information during storage and transmission.

Key Features

- Ensures data confidentiality
- Prevents unauthorized access
- Supports authentication and integrity verification
- Protects data during communication

Advantages

- Strong data protection
- Secure communication
- Prevents data theft

Limitations

- Requires proper key management
- Can become vulnerable if weak algorithms are used

Relevance to Project

Cryptography is used as the first layer of security in the proposed system by encrypting secret messages before they are hidden inside images.

Advanced Encryption Standard (AES)

AES (Advanced Encryption Standard) is one of the most widely used symmetric encryption algorithms. It was developed by NIST and has become an international standard for securing digital information.

Key Features

- Symmetric-key encryption algorithm
- Supports 128-bit, 192-bit, and 256-bit keys
- Fast and efficient
- Resistant to known practical attacks

Advantages

- High security level
- Fast execution speed
- Widely accepted industry standard

Limitations

- Secure key exchange is required
- Both sender and receiver must possess the same key

Relevance to Project

This project uses AES-128 in CBC mode to encrypt secret messages before embedding them into images.

Steganography

Steganography is the process of concealing information inside another digital medium such as images, audio files, videos, or text documents. Unlike cryptography, it hides the existence of the message itself.

Key Features

- Conceals secret information
- Uses multimedia files as carriers
- Makes communication less noticeable

Advantages

- Hidden communication
- Additional security layer
- Difficult for ordinary users to detect

Limitations

- Hidden data may be destroyed by image compression
- Vulnerable to steganalysis techniques

Relevance to Project

The project uses image steganography to conceal encrypted messages inside digital images.

Least Significant Bit (LSB) Steganography

LSB is one of the most popular image steganography techniques. It works by replacing the least significant bits of image pixels with secret message bits.

Key Features

- Simple implementation
- Minimal image distortion
- High embedding capacity
- Fast processing

Advantages

- Preserves image quality
- Easy to implement
- Efficient for text hiding

Limitations

- Sensitive to image modifications
- Can be detected using advanced steganalysis methods

Relevance to Project

LSB steganography is used to embed encrypted messages into PNG images without causing visible changes.

Image Encryption

Image encryption is the process of converting a normal image into an unreadable form to prevent unauthorized viewing.

Common Techniques

- Pixel permutation
- Row and column swapping
- XOR operations
- Chaotic encryption methods

Advantages

- Protects visual information
- Prevents unauthorized access
- Enhances privacy

Limitations

- Requires correct decryption key
- Additional processing overhead

Relevance to Project

The proposed system encrypts images using row swapping, column swapping, and XOR operations before decryption restores the original image.

User Authentication Systems

User authentication is a mechanism used to verify the identity of users before granting access to a system.

Key Features

- User registration
- Login verification
- Password management
- Access control

Advantages

- Prevents unauthorized access

- Improves system security
- Supports user accountability

Limitations

- Weak passwords can reduce security
- Requires secure credential storage

Relevance to Project

The project includes a login and registration system to ensure that only authorized users can access security features.

SQLite Database Management

SQLite is a lightweight relational database management system that stores data in a single file without requiring a dedicated database server.

Key Features

- Lightweight
- Serverless architecture
- Easy integration with Python
- Efficient data storage

Advantages

- Simple deployment
- Low resource consumption
- Reliable data management

Limitations

- Not suitable for large-scale distributed systems
- Limited concurrent access

Relevance to Project

SQLite is used to store user credentials and manage authentication information securely.

SECURE STEGANOGRAPHY SYSTEM

Project Objective

In the modern digital era, information security has become one of the most important concerns. Sensitive data transmitted over networks can easily become vulnerable to unauthorized access, modification, or theft. Traditional encryption secures data content but does not hide the existence of the message itself. Steganography, on the other hand, conceals the presence of secret data within multimedia files such as images.

This project, "Secure Steganography System," combines cryptography and steganography to provide dual-layer security. The system encrypts messages using the Advanced Encryption Standard (AES) algorithm and then hides the encrypted message inside digital images using Least Significant Bit (LSB) steganography techniques.

In addition to text hiding, the system also provides image encryption functionality using row-column swapping and XOR operations. A secure login system, key persistence, metadata handling, and integrity verification mechanisms further enhance the overall security of the application.

Problem Statement

Many communication systems only rely on encryption, which protects message content but still exposes the existence of sensitive communication. Attackers may still detect encrypted files andattempt brute-force or cryptanalysis attacks.

The problem addressed in this project is to create a secure system that:

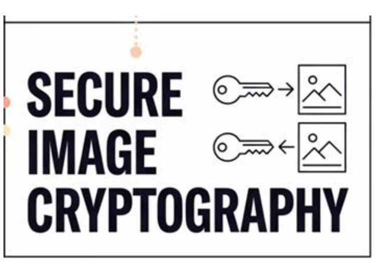
- Encrypts confidential data
- Hides encrypted data within images
- Prevents unauthorized access
- Verifies data integrity
- Provides secure image encryption and decryption

Objectives

The major objectives of this project are:

- To implement secure user authentication
- To encrypt secret messages using AES encryption
- To hide encrypted messages inside images
- To retrieve and decrypt hidden messages securely
- To encrypt and decrypt images using image processing techniques
- To verify integrity and detect tampering
- To provide metadata and key management

A System that Secure data through



Scope of the Project

The scope of this project includes:

- Secure message hiding in PNG images
- AES-based text encryption
- Image encryption and decryption
- User registration and login system
- Integrity verification using SHA-256 hashing

Existing System

Existing steganography systems often:

- Hide plain text without encryption
- Lack integrity verification
- Do not provide user authentication
- Are vulnerable to unauthorized extraction
- Do not support image encryption
- These limitations reduce the overall security of communication.
- Detects modified or corrupted messages

Proposed System

- AES encryption for message confidentiality
- LSB steganography for hidden communication
- SHA-256 hashing for integrity verification
- SQLite-based login system
- Image encryption using row/column swapping and XOR operations
- Metadata persistence for secure decryption

Features of the System

Main Features

User Authentication

- User registration
- Secure password hashing using SHA-256
- Login verification using SQLite database

Steganography

- Hides encrypted messages inside images
- Uses Least Significant Bit (LSB) encoding
- Supports extraction and decryption

AES Encryption

- AES-CBC mode encryption
- Random key generation
- Key saving and loading functionality

Image Encryption

- Row swapping
- Column swapping
- XOR-based encryption
- Metadata generation and storage

Integrity Verification

- SHA-256 hashing
- Detects modified or corrupted messages

Metadata Handling

- Stores swap operations
- Verifies encryption key correctness

Technologies and Tools Used

Technology / Tool	Purpose
Python	Main programming language
JupyterLab	Development environment
SQLite3	User database management
OpenCV (cv2)	Image processing
NumPy	Numerical operations
Pillow (PIL)	Image manipulation
PyCryptodome	AES encryption
Matplotlib	Visualization
SHA-256	Password and integrity hashing

Methodology

The project follows the following methodology:

1. User registers and logs into the system.
2. AES encryption key is generated.
3. Secret message is encrypted using AES-CBC.
4. Encrypted data is hidden inside an image using LSB steganography.
5. Image capacity is checked before embedding.
6. Receiver extracts encrypted data from image.
7. AES decryption retrieves original message.
8. SHA-256 verifies message integrity.
9. For image encryption:
    - Row swaps are applied
    - Column swaps are applied
    - XOR encryption is performed
10. Metadata is stored for decryption purposes.

System Architecture

Workflow: Embedding Path

Input
Raw secret message provided by user

Auth
Secure login & identity check

AES encrypt
Convert to cipher via AES-CBC

LSB Hide
Embed bits into cover image pixels

Stego img
Final output within hidden data

Workflow: Extraction Path

Stego Input
Receiver provides the encoded image.

LSB Extract
Bits pulled from least Significant positions.

AES Decrypt
Ciphertext reverted using session key.

Integrity
Hash check via SHA-256 algorithm.

Original
Secure message restored to

plaintext.

Workflow: Image Scrambling

XOR Logic
Bitwise encryption with random mask.

Original
High-resolution source image file.

Row Swap
Spatial reordering of pixel Rows.

-

Ecrypted
Visual noise for unauthorized viewers.

Modules Description

Module 1: Login System

This module handles:

- User registration
- Password hashing
- Login authentication
- Database management

```
==============================
  SECURE STEGANOGRAPHY SYSTEM
==============================
1. Login
2. Register
3. Exit
Enter choice:  2
Choose username:  Mariam
Choose password:  mariam123
✓ Registration successful!
```

```
==============================
  SECURE STEGANOGRAPHY SYSTEM
==============================
1. Login
2. Register
3. Exit
Enter choice:  1
Username:  Mariam
Password:  mariam123

✓ Welcome, Mariam!
```

Module 2: AES Encryption Module

Responsible for:

- AES key generation
- Message encryption
- Message decryption
- Key storage/loading

```
==============================
 SECURE STEGANOGRAPHY SYSTEM
==============================
1.  Generate AES Key
2.  Save AES Key to File
3.  Load AES Key from File
4.  Encrypt & Hide Message
5.  Extract & Decrypt Message
6.  Encrypt Image
7.  Decrypt Image
8.  Exit

Enter choice:  1

✓ AES Key Generated:
Z6ErxAGPp82XbZZx8TFjAw==
→ Use Option 2 to save it to a file!
```

```
==============================
 SECURE STEGANOGRAPHY SYSTEM
==============================
1.  Generate AES Key
2.  Save AES Key to File
3.  Load AES Key from File
4.  Encrypt & Hide Message
5.  Extract & Decrypt Message
6.  Encrypt Image
7.  Decrypt Image
8.  Exit

Enter choice:  2
Filename (default: aes_key.key):
✓ Key saved to aes_key.key
```

Module 3: Steganography Module

Handles:

- Binary conversion
- LSB embedding
- Data extraction
- Capacity checking

```
# ========================
# STEGANOGRAPHY FUNCTIONS
# ========================
END_MARK = "<END>"
SEP = "<SEP>"

def text_to_binary(text):
    return ''.join(format(ord(c), '08b') for c in text)


# ========================
# STEP 5: CAPACITY CHECK
# ========================
def get_image_capacity(image_path):
    img = Image.open(image_path).convert("RGB")
    max_bytes = (img.width * img.height * 3) // 8
    return max_bytes

def get_meta_filename(image_path):
    base = image_path.rsplit(".", 1)[0]
    return base + "_meta.json"
```

Module 4: Image Encryption Module

Performs:

- Row swapping
- Column swapping
- XOR encryption

Step	Operation
Row Swaps	Randomly swap pairs of rows using a seeded PRNG (seed = key)
Column Swaps	Randomly swap pairs of columns (seed = key + 999)
XOR Stream	Generate a key-seeded NumPy random matrix; XOR with pixel array

Module 5: Metadata Module

Responsible for:

- Metadata storage
- Key verification
- Swap operation persistence

Algorithms Used

AES (Advanced Encryption Standard)

- Symmetric encryption algorithm
- Uses CBC mode
- Provides strong confidentiality

LSB Steganography

- Stores secret bits inside least significant bits of pixels
- Minimizes visible image distortion

SHA-256 Hashing

- Generates secure hash values
- Used for passwords and integrity verification

XOR Encryption

- Pixel values XORed with random keystream
- Provides image scrambling

Implementation Details

The project is implemented in Python using multiple libraries.

Key Functions

Authentication Functions

- register_user()
- login_user()

AES Functions

- encrypt_message()
- decrypt_message()

Steganography Functions

- encode_image()
- decode_image()

Image Encryption Functions

- encrypt_image()
- decrypt_image()

Metadata Functions

- save_meta()
- load_meta()

Security Mechanisms

The system uses multiple security layers:

Confidentiality

Provided by AES encryption.

Authentication

User login system with hashed passwords.

Integrity

SHA-256 hash comparison verifies message correctness.

Hidden Communication

LSB steganography conceals encrypted data.

Key Verification

Wrong decryption keys are detected before image restoration.

Testing and Results

The system was tested successfully for:

- User registration/login
- AES key generation
- Secure message hiding
- Message extraction and decryption
- Integrity verification
- Image encryption/decryption

- Wrong key detection

Results showed:

- Hidden messages were successfully retrieved
- Integrity verification worked correctly
- Encrypted images were restored successfully

Advantages of the System

- Dual-layer security
- Secure authentication
- Hidden communication
- Strong encryption
- Tamper detection
- Easy to use
- Secure image encryption
- Metadata persistence

Limitations

- Supports mainly PNG images
- Large messages require larger images
- Console-based interface only
- No cloud or network communication support

Future Enhancements

Future improvements may include:

- GUI-based interface
- Support for audio/video steganography
- Cloud storage integration
- Multi-user communication

- RSA-based key exchange
- AI-based steganalysis resistance
- Mobile application support

Conclusion

The Secure Steganography System successfully combines cryptography and steganography to provide enhanced information security. The project demonstrates how encrypted messages can be securely hidden within images while maintaining confidentiality, integrity, and authentication.

The integration of AES encryption, SHA-256 hashing, LSB steganography, and image encryption techniques creates a powerful multi-layer security system suitable for secure digital communication and academic research purposes.

References

1. William Stallings – Cryptography and Network Security
2. Python Official Documentation
3. PyCryptodome Documentation
4. OpenCV Documentation
5. Research Papers on St`eganograp